CAHIER DES CHARGES TECHNIQUE PLATEFORME D'E-COMMERCE D'E-BOOKS

PROJET : [Nom de la plateforme]
VERSION : 1.0
DATE : 03 Août 2026
DESTINATAIRE : Equipe de développement

TABLE DES MATIÈRES

1. PRESENTATION DU PROJET 6
   1.1 Contexte 6
   1.2 Public cible 6
   1.3 Zone géographique 6
   1.4 Monétisation 6
2. VISION ET OBJECTIFS 6
   2.1 Vision 6
   2.2 Objectifs stratégiques 7
   2.3 Objectifs techniques 7
3. ARCHITECTURE TECHNIQUE GLOBALE 7
   3.1 Stack technologique 7
   3.2 Architecture globale (schéma) 8
   3.3 Schéma de déploiement 8
4. PHASE 1 — SITE E-COMMERCE ADMINISTRÉ (MVP) 8
   4.1 PORTAIL CLIENT (FRONTEND — Next.js) 9
      4.1.1 Page d'accueil 9
      4.1.2 Catalogue / Page de recherche 9
      4.1.3 Fiche produit (e-book) 9
      4.1.4 Panier 9
      4.1.5 Processus de paiement (Checkout) 10
      4.1.6 Espace client / Bibliothèque 10
      4.1.7 Lecteur ePub intégré (Web Reader) 10
      4.1.8 Pages statiques 11
      4.1.9 Composants UI réutilisables 11
   4.2 BACK-OFFICE ADMINISTRATEUR 11
      4.2.1 Authentification 11
      4.2.2 Tableau de bord (Dashboard) 12
      4.2.3 Gestion du catalogue (CRUD E-books) 12
      4.2.4 Gestion des auteurs 12
      4.2.5 Gestion des catégories 13
      4.2.6 Gestion des commandes 13
      4.2.7 Gestion des clients 13
      4.2.8 Gestion des avis 13
      4.2.9 Gestion des promotions 13
      4.2.10 Gestion des contenus (CMS) 14
      4.2.11 Rapports et analytics 14
      4.2.12 Paramètres système 14

4.3 API BACKEND (PYTHON — FastAPI) 14
4.3.1 Architecture 14
4.3.2 Modules métier 14
4.3.3 Middleware et sécurité 16
4.4 BASE DE DONNÉES (POSTGRESQL) 16
4.4.1 Modèle Conceptuel de Données (MCD) 16
4.4.2 Indexation et optimisation 17
4.4.3 Contraintes d'intégrité 17
4.5 INTÉGRATION MOBILE MONEY 18
4.5.1 Opérateurs supportés (Phase 1) 18
4.5.2 Flux de paiement 18
4.5.3 Architecture de paiement 18
4.5.4 Gestion des erreurs 18
4.5.5 Sécurité 18
4.6 SÉCURITÉ & CONFORMITÉ 19
4.6.1 Protection des données (RGPD) 19
4.6.2 Sécurité technique 19
4.6.3 Conformité légale 19
4.7 LIVRAISON DES E-BOOKS 20
4.7.1 Stockage des fichiers 20
4.7.2 Téléchargement sécurisé 20
4.7.3 Lecteur intégré 20
4.7.4 Formats supportés 20
5. PHASE 2 — PLATEFORME D'AUTO-PUBLICATION 20
5.1 PORTAIL AUTEUR (FRONTEND) 21
5.1.1 Inscription et authentification 21
5.1.2 Tableau de bord auteur (Dashboard) 21
5.1.3 Gestion des publications (CRUD Livres) 21
5.1.4 Statistiques détaillées par livre 23
5.1.5 Paiements et revenus 23
5.1.6 Communauté et ressources 23
5.2 WORKFLOW DE PUBLICATION 24
5.2.1 Processus de validation 24
5.2.2 Critères de validation 24
5.2.3 Outils de validation automatique (pré-contrôle) 24
5.3 TABLEAU DE BORD AUTEUR (ANALYTICS) 24
5.3.1 Vue d'ensemble (Dashboard principal) 24
5.3.2 Rapports détaillés 25
5.3.3 Notifications en temps réel 25
5.4 SYSTÈME DE REVENUS ET PAIEMENTS 26

5.4.1 Calcul des revenus 26
5.4.2 Paiement automatique mensuel 26
5.4.3 Gestion des remboursements 26
5.4.4 Fiscalité 26
5.5 VALIDATION ÉDITORIALE (BACK-OFFICE ADMIN) 27
5.5.1 File de validation 27
5.5.2 Interface de validation 27
5.5.3 Motifs de refus prédéfinis 27
5.5.4 Journal de validation 28
6. SPÉCIFICATIONS TECHNIQUES DÉTAILLÉES 28
6.1 MODÈLE DE DONNÉES COMPLET (PHASE 1 + PHASE 2) 28
6.1.1 Table USERS 28
6.1.2 Table AUTHORS (profil auteur, Phase 2) 28
6.1.3 Table BOOKS 28
6.1.4 Table CATEGORIES 29
6.1.5 Table BOOK_CATEGORIES (association N-N) 29
6.1.6 Table ORDERS 29
6.1.7 Table ORDER_ITEMS 30
6.1.8 Table PAYMENTS 30
6.1.9 Table COUPONS 30
6.1.10 Table REVIEWS 31
6.1.11 Table LIBRARY (achats client) 31
6.1.12 Table READING_PROGRESS 31
6.1.13 Table BOOKMARKS 31
6.1.14 Table AUTHOR_PAYMENTS (Phase 2) 32
6.1.15 Table AUDIT_LOGS 32
6.2 API REST — ENDPOINTS DÉTAILLÉS 32
6.2.1 Authentification 32
6.2.2 Livres (Catalogue Public) 33
6.2.3 Avis 33
6.2.4 Panier 33
6.2.5 Commandes 34
6.2.6 Paiements 34
6.2.7 Bibliothèque (Library) 34
6.2.8 Progression de lecture 34
6.2.8 Marque-pages 35
6.2.9 Admin — Gestion du catalogue 35
6.2.10 Admin — Gestion des commandes 35
6.2.11 Admin — Rapports 35
6.2.12 Auteur — Gestion des publications (Phase 2) 36

6.2.13 Auteur — Statistiques (Phase 2) 36
6.2.14 Admin — Validation éditoriale (Phase 2) 36
6.3 INTÉGRATIONS TIERCES 37
6.3.1 Mobile Money APIs 37
6.3.2 Stockage de fichiers (S3-compatible) 37
6.3.3 Email (Transactionnels) 37
6.3.4 Monitoring et Observabilité 38
6.3.5 CI/CD 38
6.4 PERFORMANCE & SCALABILITÉ 38
6.4.1 Cache 38
6.4.2 Base de données 38
6.4.3 CDN et assets 39
6.4.4 Upload de fichiers 39
6.4.5 Scalabilité horizontale 39
7. EXIGENCES NON-FONCTIONNELLES 39
7.1 Disponibilité 39
7.2 Performance 39
7.3 Sécurité 40
7.4 Accessibilité 40
7.5 Internationalisation (i18n) 40
7.6 Compatibilité navigateurs 40
8. LIVRABLES ET PLANNING 40
8.1 Phase 1 — MVP E-commerce (3-4 mois) 40
8.2 Phase 2 — Auto-publication (4-5 mois après Phase 1) 41
8.3 Livrables documentaires 41
9. GLOSSAIRE 41
ANNEXES 43
ANNEXE A : STACK TECHNIQUE DÉTAILLÉE 43
ANNEXE B : MODÈLE DE DONNÉES VISUEL (DIAGRAMME ER) 43
ANNEXE C : FLUX DE PAIEMENT MOBILE MONEY (SÉQUENCE) 44
ANNEXE D : CALCUL DE RÉMUNÉRATION AUTEUR 44
ANNEXE E : CHECKLIST DE SÉCURITÉ 44

1. PRESENTATION DU PROJET

1.1 Contexte

Le projet consiste en la création d'une plateforme d'e-commerce d'e-books destinée aux marchés francophones et subsahariens d'Afrique. La plateforme visera à concurrencer les acteurs existants tels que Kobo, Amazon KDP, en proposant une solution adaptée aux spécificités locales, notamment le paiement via Mobile Money.

1.2 Public cible

- Phase 1 : Lecteurs africains francophones (B2C)
- Phase 2 : Auteurs indépendants, auto-éditeurs et éditeurs (B2B2C)

1.3 Zone géographique

Pays francophones et subsahariens d'Afrique prioritaires :

- Sénégal,
- Côte d'Ivoire,
- Mali,
- Burkina Faso,
- Niger,
- Bénin,
- Togo,
- Cameroun,
- Gabon,
- Congo,
- RDC,
- Burundi,
- Madagascar,
- Maurice,
- Comores,
- Tchad,
- Guinée et
- Mauritanie.

1.4 Monétisation

- Commission perçue par la plateforme : 30 % du prix de vente
- Revenus auteur : 70 % du prix de vente
- Paiement automatique aux auteurs : fin de chaque mois
- Seuil minimum de paiement : 10 000 FCFA (ou équivalent local)

2. VISION ET OBJECTIFS

2.1 Vision

Devenir la plateforme de référence pour la distribution d'e-books en Afrique francophone, en offrant une expérience fluide tant pour les lecteurs que pour les auteurs, avec un modèle économique équitable et transparent.

2.2 Objectifs stratégiques

- Lancer un MVP e-commerce en 3-4 mois (Phase 1)
- Atteindre 1 000 ventes/mois d'ici 6 mois après le lancement
- Intégrer l'auto-publication après 6 mois d'observation (Phase 2)
- Atteindre 500 auteurs inscrits dans les 12 mois suivant la Phase 2
- Couvrir 15 pays africains francophones d'ici 24 mois

2.3 Objectifs techniques

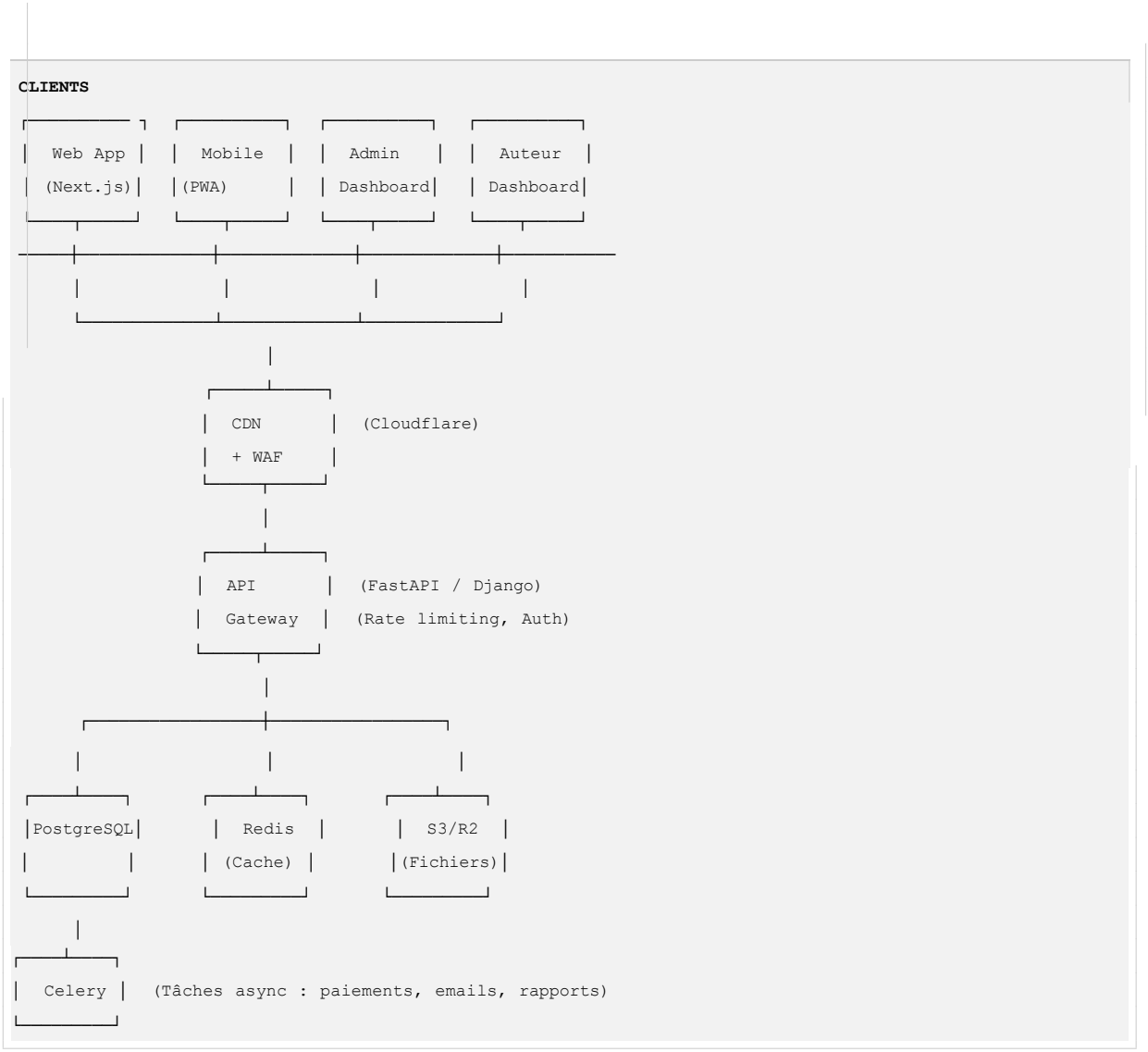
- Disponibilité : 99.9 % SLA
- Temps de réponse API : < 200 ms (p95)
- Temps de chargement page : < 2 s (Lighthouse Performance > 90)
- Support : 10 000 utilisateurs simultanés (Phase 1)
- Stockage : 1 To d'e-books (Phase 1), évolutif à 10 To (Phase 2)

3. ARCHITECTURE TECHNIQUE GLOBALE

3.1 Stack technologique

Couche	Technologie	Justification
Frontend	Next.js 14 (App Router) + React 18	SSR, SEO, performance
	TypeScript	Typage fort, maintenabilité
	Tailwind CSS + shadcn/ui	Design system cohérent
	React Query (TanStack Query)	Gestion état serveur
	Zustand	Gestion état client
Backend	Python 3.11+	Ecosystème mature, IA
	FastAPI (ou Django REST Framework)	Performance, async
	Celery + Redis	Tâches asynchrones
	SQLAlchemy (ou Django ORM)	ORM robuste
Base de données	PostgreSQL 15+	ACID, JSON, full-text
	Redis	Cache, sessions, queue
Stockage	AWS S3 / Cloudflare R2	Stockage fichiers ePub
	CloudFront / Cloudflare CDN	Distribution globale
Paiement	API Mobile Money (Orange Money, MTN, MoovMoney, MixByYass, etc.)	Paiement local dominant
	Stripe (futur)	Cartes bancaires
Authentification	OAuth 2.0 / JWT	Sécurité, sessions
	Clerk / Auth0 (option)	Gestion identité rapide
Email	Resend / Mailgun / Brevo	Transactionnels
Monitoring	Sentry + Grafana + Prometheus	Observabilité
CI/CD	GitHub Actions	Déploiement automatisé
Hébergement	Railway / Render / AWS ECS	Scalabilité, coûts

3.2 Architecture globale (schéma)



3.3 Schéma de déploiement

- Environnements : Développement → Staging → Production
- Conteneurisation : Docker + Docker Compose (dev) / Kubernetes (prod)
- Base de données : PostgreSQL managed (Supabase / Railway / AWS RDS)
- Stockage : Cloudflare R2 (pas de frais de sortie, compatible S3)

4. PHASE 1 — SITE E-COMMERCE ADMINISTRÉ (MVP)

- Durée estimée : 3-4 mois
- Objectif : Valider le marché avec un catalogue géré par l'administrateur

4.1 PORTAIL CLIENT (FRONTEND — Next.js)

4.1.1 Page d'accueil

- Hero section avec bannière promotionnelle (carrousel)
- Section "Nouveautés" (8 livres, pagination infinie)
- Section "Best-sellers" (top 10 du mois)
- Section "Catégories populaires" (grille visuelle)
- Section "Recommandations pour vous" (si connecté)
- Barre de recherche en header (recherche full-text)
- Footer : liens légaux, réseaux sociaux, newsletter

4.1.2 Catalogue / Page de recherche

- Filtres : Catégorie, Prix (min/max), Note, Langue, Date de publication
- Tri : Pertinence, Prix croissant/décroissant, Date, Popularité
- Affichage : Grille (3-4 colonnes) ou Liste
- Pagination : Infinie ou numérotée
- Compteur de résultats
- Facettes dynamiques (mise à jour selon filtres actifs)

4.1.3 Fiche produit (e-book)

- Couverture HD (zoom au survol)
- Titre, sous-titre, auteur(s) avec lien vers page auteur
- Prix affiché en devise locale (FCFA, XOF, etc.)
- Note moyenne + nombre d'avis (étoiles)
- Résumé / Description (HTML enrichi)
- Détails techniques : Format (ePub), Nombre de pages, ISBN, Langue
- Tags / Mots-clés
- Bouton "Acheter maintenant" (redirection paiement)
- Bouton "Ajouter au panier"
- Section "Avis clients" (liste paginée, formulaire d'ajout si connecté)
- Section "Livres similaires" (recommandation basée sur catégorie)
- Section "Du même auteur"
- Partage social (WhatsApp, Facebook, Twitter/X, Copier le lien)
- Métadonnées SEO riches (Schema.org Book)

4.1.4 Panier

- Liste des articles avec image, titre, prix unitaire, quantité
- Modification de quantité (1-5 max)

- Suppression d'article
- Calcul du sous-total
- Champ code promo (avec validation et affichage de réduction)
- Estimation des frais (si applicable)
- Bouton "Passer la commande"
- Sauvegarde du panier en localStorage (si non connecté)

4.1.5 Processus de paiement (Checkout)

- Étape 1 : Informations personnelles (nom, email, téléphone)
- Étape 2 : Récapitulatif de la commande
- Étape 3 : Choix du mode de paiement
- Mobile Money (Orange Money, MTN Mobile Money, Wave, MoovMoney, MixByYass etc.)
- Affichage du numéro de paiement et instructions
- Validation automatique via webhook API opérateur
- Étape 4 : Confirmation de commande + téléchargement
- Page de succès avec récapitulatif et liens de téléchargement
- Email de confirmation automatique

4.1.6 Espace client / Bibliothèque

- Tableau de bord : statistiques rapides (livres achetés, dépenses)
- Bibliothèque : grille de tous les e-books achetés
- Couverture, titre, auteur, date d'achat
- Bouton "Télécharger" (lien signé, expiration 15 min, 5 téléchargements max)
- Compteur de téléchargements restants
- Bouton "Lire en ligne" (lecteur ePub intégré)
- Historique des commandes (liste avec détails, factures PDF)
- Paramètres du compte : nom, email, téléphone, mot de passe
- Préférences : langue, notifications

4.1.7 Lecteur ePub intégré (Web Reader)

- Basé sur EPUB.js ou Readium
- Navigation : chapitres, pages, barre de progression
- Personnalisation : taille police, interligne, thème (clair/sépia/sombre)
- Marque-pages et annotations (si connecté)

- Mode plein écran
- Synchronisation de la position de lecture (si connecté)
- Mode hors ligne (PWA + Service Worker + Cache des e-books)

4.1.8 Pages statiques

- À propos
- Comment ça marche (guide d'achat)
- FAQ
- Conditions générales de vente
- Politique de confidentialité (RGPD)
- Contact (formulaire + email)
- Blog (CMS headless ou markdown)

4.1.9 Composants UI réutilisables

- Header : logo, recherche, navigation, panier, compte
- Footer : liens, newsletter, réseaux sociaux
- Carte livre (BookCard) : couverture, titre, auteur, prix, note
- Carrousel (pour hero et sections)
- Modal / Dialog
- Toast notifications
- Skeleton loaders (état de chargement)
- Pagination
- Breadcrumbs
- StarRating (affichage et saisie)

4.2 BACK-OFFICE ADMINISTRATEUR

4.2.1 Authentification

- Login sécurisé (2FA recommandé)
- Gestion des rôles : Super Admin, Admin, Modérateur
- Journal d'audit des actions administratives

4.2.2 Tableau de bord (Dashboard)

KPIs en temps réel :

- Ventes du jour / semaine / mois / année
- Revenus totaux et commission plateforme
- Nombre de nouveaux clients
- Panier moyen
- Taux de conversion
- Livres les plus vendus (top 10)
- Graphiques : courbes de ventes, répartition par catégorie, par pays
- Alertes : stocks faibles (si applicable), validations en attente

4.2.3 Gestion du catalogue (CRUD E-books)

- Liste des livres avec filtres et recherche
- Création / Édition d'un livre :
  - Titre, sous-titre, slug (auto-généré)
  - Auteur(s) (création rapide si nouvel auteur)
  - Description (éditeur WYSIWYG)
  - Résumé court (meta description)
  - Catégorie(s) (arborescence)
  - Tags / Mots-clés
  - Prix (en FCFA par défaut, avec conversion automatique)
  - ISBN (optionnel)
  - Langue
  - Nombre de pages
  - Date de publication
  - Upload couverture (JPG/PNG, min 800x1200px, max 5Mo)
  - Upload fichier ePub (validation format, max 50Mo)
  - Extrait gratuit (premiers chapitres, upload ePub séparé)
  - Statut : Brouillon, En attente, Publié, Archivé
  - Duplication de livre (pour variantes)
  - Export CSV / Import CSV (batch)
  - Prévisualisation de la fiche produit

4.2.4 Gestion des auteurs

- CRUD auteur : nom, bio, photo, email, site web, réseaux sociaux
- Association avec les livres

- Statistiques par auteur (ventes, revenus)

4.2.5 Gestion des catégories

- Arborescence infinie (parent/enfant)
- Slug, description, image
- Ordre d'affichage
- Nombre de livres par catégorie

4.2.6 Gestion des commandes

- Liste avec filtres (statut, date, client, montant)
- Statuts : En attente, Payée, En cours, Livrée, Annulée, Remboursée
- Détail de commande : articles, client, paiement, historique
- Actions : Valider, Annuler, Rembourser (partiel ou total)
- Génération de facture PDF
- Export CSV des commandes

4.2.7 Gestion des clients

- Liste avec recherche et filtres
- Fiche client : infos, historique d'achats, bibliothèque
- Actions : désactiver compte, envoyer email
- Export CSV

4.2.8 Gestion des avis

- Liste des avis avec modération
- Actions : Approuver, Rejeter, Répondre
- Filtrage par note, statut, livre
- Alertes sur avis signalés

4.2.9 Gestion des promotions

- Codes promo : code, type (% ou montant fixe), valeur, date validité, nombre d'utilisations max, usage unique par client, livres éligibles
- Promotions automatiques : "3 pour 2", "-20% sur la catégorie X"
- Historique d'utilisation des codes

4.2.10 Gestion des contenus (CMS)

- Pages statiques (À propos, CGV, etc.)
- Bannières promotionnelles (position, dates, ciblage)
- Blog : articles avec éditeur riche, SEO
- Newsletter : rédaction, envoi, statistiques (ouverture, clic)

4.2.11 Rapports et analytics

- Rapport de ventes (par période, par livre, par catégorie, par pays)
- Rapport de revenus (commission plateforme vs auteur)
- Rapport de clients (acquisition, rétention)
- Export Excel/PDF
- Programma-tion de rapports automatiques (email)

4.2.12 Paramètres système

- Configuration des devises (FCFA par défaut, taux de change)
- Configuration des frais de commission (30% / 70%)
- Configuration des seuils de paiement
- Configuration des emails (templates, expéditeur)
- Configuration des intégrations Mobile Money
- Gestion des administrateurs et rôles • Logs système

4.3 API BACKEND (PYTHON — FastAPI)

4.3.1 Architecture

- Pattern : Clean Architecture / Hexagonal (ports & adapters)
- Structure : app/ api/ # Routes FastAPI v1/ auth.py books.py orders.py payments.py users.py admin.py core/ # Config, sécurité, dépendances models/ # Modèles SQLAlchemy schemas/ # Pydantic (validation/sérialisation) services/ # Logique métier repositories/ # Accès données (abstraction DB) tasks/ # Tâches Celery utils/ # Helpers main.py

4.3.2 Modules métier

MODULE AUTHENTIFICATION

- POST /api/v1/auth/register
- POST /api/v1/auth/login

- POST /api/v1/auth/refresh
- POST /api/v1/auth/logout
- POST /api/v1/auth/forgot-password
- POST /api/v1/auth/reset-password
- POST /api/v1/auth/verify-email
- GET /api/v1/auth/me
- PUT /api/v1/auth/me

MODULE LIVRES (CATALOGUE PUBLIC)

- GET /api/v1/books — Liste avec filtres, tri, pagination
- GET /api/v1/books/{slug} — Détail d'un livre
- GET /api/v1/books/{slug}/related — Livres similaires
- GET /api/v1/books/search — Recherche full-text (PostgreSQL tsvector)
- GET /api/v1/ categories — Liste des catégories (arborescence)
- GET /api/v1/categories/{slug}/books — Livres par catégorie

MODULE AVIS

- GET /api/v1/books/{slug}/reviews
- POST /api/v1/books/{slug}/reviews (au-thentifié)
- PUT /api/v1/reviews/{id} (auteur de l'avis)
- DELETE /api/v1/reviews/{id} (auteur ou admin)

MODULE PANIER

- GET /api/v1/cart
- POST /api/v1/cart/items — Ajouter un article
- PUT / api/v1/cart/items/{id} — Modifier quantité
- DELETE /api/v1/cart/items/{id} — Supprimer article
- POST /api/v1/cart/apply-coupon
- DELETE /api/v1/cart/clear

MODULE COMMANDES

- POST /api/v1/orders — Créer une commande
- GET /api/v1/orders — Liste (client)
- GET /api/v1/orders/{id} — Détail
- POST /api/v1/orders/{id}/cancel

MODULE PAIEMENTS

- POST /api/v1/payments/initiate — Initier paiement Mobile Money
- POST /api/v1/payments/webhook/{provider} — Webhook confirmation
- GET /api/v1/payments/{id}/sta-tus

MODULE BIBLIOTHÈQUE (CLIENT)

- GET /api/v1/library — Livres achetés
- GET /api/v1/ library/{book_id}/download — Lien signé temporaire
- GET /api/v1/library/{book_id}/read — URL lecteur intégré

MODULE ADMIN (PROTÉGÉ PAR RÔLE)

- CRUD complet sur toutes les entités
- Endpoints spécifiques admin (rapports, statistiques, modération)

4.3.3 Middleware et sécurité

- CORS configuré (origines whitelistées)
- Rate limiting par IP et par utilisateur (Redis)
- Validation des entrées (Pydantic)
- Protection CSRF
- Headers de sécurité (HSTS, X-Frame-Options, etc.)
- Logging de toutes les requêtes

4.4 BASE DE DONNÉES (POSTGRESQL)

4.4.1 Modèle Conceptuel de Données (MCD)

ENTITÉS PRINCIPALES :

USER BOOK CATEGORY

id(PK) id (PK) id (PK) email (UQ) title name password_hash slug (UQ) slug (UQ) first_name description description last_name price parent_id phone isbn image_url avatar_url language sort_order role page_count is_active is_active cover_url created_at email_verified file_url updated_at created_at excerpt_url updated_at status author_ category_id is_featured published_at created_at updated_at

AUTHOR BOOK CATEGORY

id (PK) (assoc) name bio book_id photo_url category_id email website social_links created_at updated_at

ORDER ORDER_ITEM

id (PK) order_id user_id book_id status quantity total_amount unit_price currency

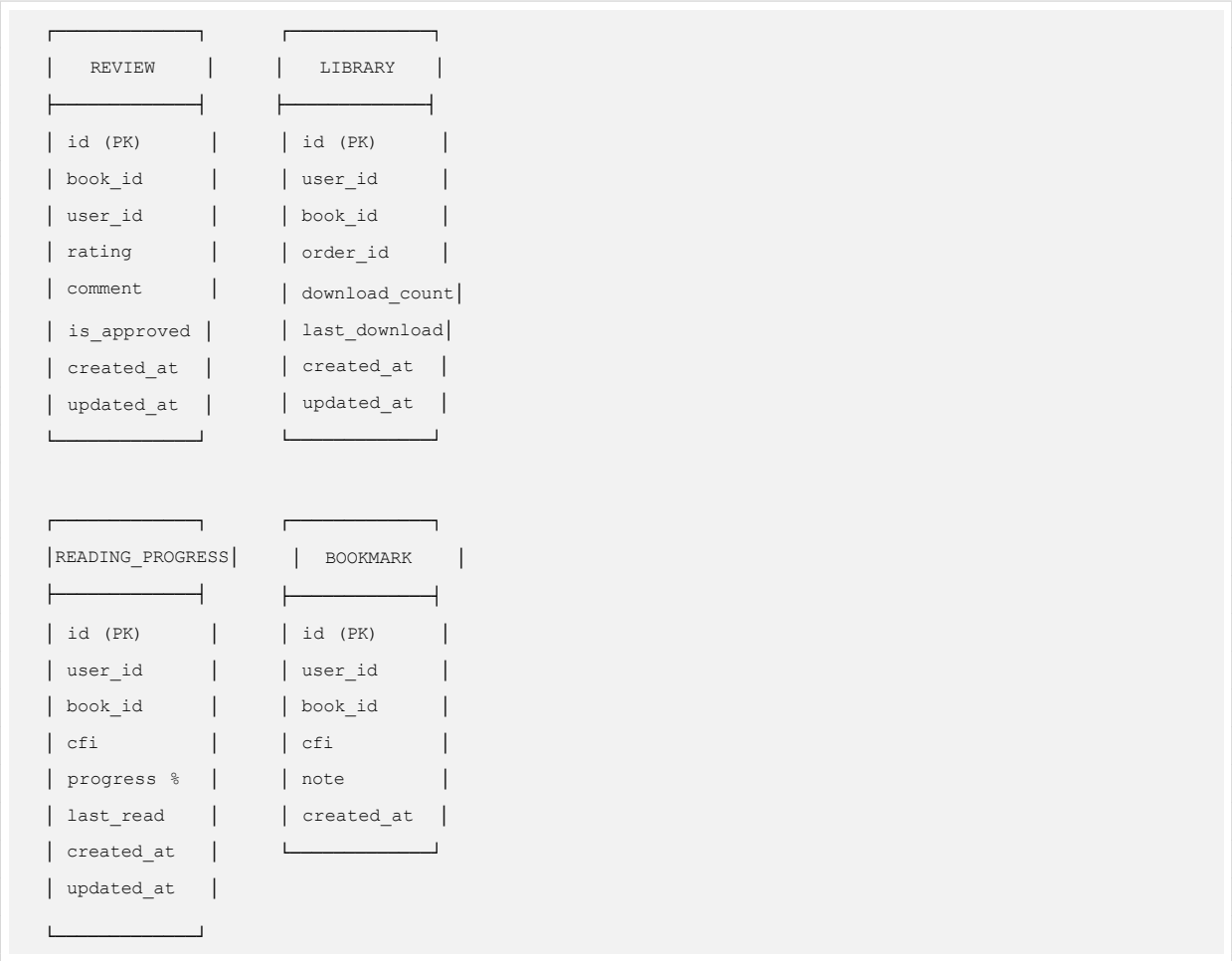
total_price payment_id created_at coupon_id created_at updated_at

PAYMENT COUPON

id (PK) id (PK) order_id code (UQ) provider type provider_ref value amount

min_amount currency max_uses status used_count phone_number valid_from

metadata valid_until paid_at is_active created_at created_at



4.4.2 Indexation et optimisation

- Index GIN sur tsvector (recherche full-text livres)
- Index B-tree sur : slug, email, isbn, status, created_at
- Index partiel sur les livres publiés (status = ‘published’)
- Partitionnement des tables orders et payments par mois (Phase 2)

4.4.3 Contraintes d’intégrité

- Clés étrangères avec ON DELETE CASCADE ou SET NULL selon cas
- CHECK constraints : prix > 0, note entre 1 et 5, quantité > 0
- UNIQUE sur : slug, email, isbn, code coupon
- NOT NULL sur champs obligatoires

4.5 INTÉGRATION MOBILE MONEY

4.5.1 Opérateurs supportés (Phase 1)

- Orange Money (Orange — Côte d'Ivoire, Sénégal, Mali, etc.)
- MTN Mobile Money (MTN — Côte d'Ivoire, Cameroun, etc.)
- Wave (Sénégal, Côte d'Ivoire, Burkina Faso)
- Moov Money (Moov — Côte d'Ivoire, Togo, etc.)
- Mixx By Yas (Yas — Togo.)

4.5.2 Flux de paiement

1. Client sélectionne "Payer par Mobile Money"
2. Saisie du numéro de téléphone et choix de l'opérateur
3. API envoie requête de paiement à l'opérateur
4. Client reçoit notification push/SMS et valide sur son téléphone
5. Opérateur envoie webhook de confirmation
6. Plateforme reçoit confirmation et marque la commande comme payée
7. Livraison automatique (email + bibliothèque)

4.5.3 Architecture de paiement

Client → API → Payment Service → Opérateur API (Orange/MTN/Wave) ↑

(Webhook callback) Webhook Handler Update Order Status Trigger Celery Tasks (Email, Library update)

4.5.4 Gestion des erreurs

- Timeout : retry avec backoff exponentiel (max 5 tentatives)
- Échec de paiement : notification client, proposition de réessayer
- Paiement en attente : statut "pending", vérification automatique toutes les 5 min
- Remboursement : API opérateur ou crédit sur compte client

4.5.5 Sécurité

- Authentification API opérateur (API Key + Signature HMAC)
- Vérification des webhooks (signature, idempotency)
- Journalisation de toutes les transactions

- Détection de fraude (montants anormaux, fréquence)

4.6 SÉCURITÉ & CONFORMITÉ

4.6.1 Protection des données (RGPD)

- Consentement explicite à l'inscription (checkboxes séparées)
- Politique de confidentialité détaillée
- Droit à l'accès : export des données personnelles (JSON)
- Droit à l'effacement : suppression du compte et données
- Droit à la portabilité
- Droit d'opposition aux communications marketing
- Durée de conservation définie (données commandes : 5 ans)
- DPO (Délégué à la Protection des Données) désigné
- Registre des traitements

4.6.2 Sécurité technique

- HTTPS obligatoire (TLS 1.3)
- Hashage mots de passe : Argon2id
- JWT : access token (15 min) + refresh token (7 jours) en HttpOnly cookies
- Protection XSS : échappement automatique, CSP headers
- Protection CSRF : tokens pour mutations
- Rate limiting : 100 req/min par IP, 1000 req/min authentifié
- Protection injection SQL : ORM + requêtes paramétrées
- Upload sécurisé : validation type MIME, scan antivirus, sandbox
- Stockage fichiers : URLs signées, expiration, pas de listing public
- Journalisation : logs de sécurité (tentatives connexion, anomalies)

4.6.3 Conformité légale

- Conditions Générales de Vente (CGV) — obligatoire
- Conditions Générales d'Utilisation (CGU)
- Politique de confidentialité (RGPD)
- Mentions légales
- Politique de remboursement (14 jours pour e-books non téléchargés)
- Politique de cookies (banniere + gestion préférences)

4.7 LIVRAISON DES E-BOOKS

4.7.1 Stockage des fichiers

- Bucket S3/R2 dédié avec versioning
- Structure : /books/{book_id}/book.epub, /books/{book_id}/ cover.jpg
- ACL : privé, accès uniquement via URLs signées
- Backup automatique (cross-region)

4.7.2 Téléchargement sécurisé

- URLs signées AWS (presigned URLs) avec expiration 15 minutes
- Limite de 5 téléchargements par achat
- Watermarking optionnel : nom + email du client intégré dans le ePub
- Journal des téléchargements (date, IP, user agent)

4.7.3 Lecteur intégré

- EPUB.js pour rendu côté client
- Chargement du fichier via URL signée (blob)
- Pas de stockage local du fichier complet (streaming)
- Synchronisation position de lecture (API)

4.7.4 Formats supportés

- ePub 3 (obligatoire)
- ePub 2 (compatibilité)
- Validation du fichier à l'upload (epubcheck)

5. PHASE 2 — PLATEFORME D'AUTO-PUBLICATION

Durée estimée : 4-5 mois après la Phase 1
Déclencheur : 6 mois d'observation du marché + validation du modèle

5.1 PORTAIL AUTEUR (FRONTEND)

5.1.1 Inscription et authentification

- Formulaire d'inscription : nom, email, téléphone, mot de passe
- Vérification email + téléphone (OTP SMS ou WHATSAPP)
- Profil auteur : nom de plume, bio, photo, site web, réseaux sociaux
- Informations bancaires / Mobile Money pour les paiements
- Documents : pièce d'identité (KYC — Know Your Customer)
- Conditions d'utilisation pour auteurs (acceptation obligatoire)

5.1.2 Tableau de bord auteur (Dashboard)

Vue d'ensemble inspirée de Kobo Writing Life et Amazon KDP :

- KPIs en temps réel :
  - Total des ventes (tous livres)
  - Revenus totaux (brut et net après commission 30%)
  - Nombre de livres publiés
  - Nombre de lecteurs
  - Note moyenne globale
- Graphiques :
  - Ventes par jour / semaine / mois / année (courbe)
  - Répartition des revenus par livre (camembert)
  - Répartition des ventes par pays (carte ou barres)
  - Évolution du nombre de lecteurs
- Alertes :
  - Nouvelle vente (notification en temps réel via WebSocket)
  - Nouvel avis
  - Validation/refus de publication
  - Paiement effectué

5.1.3 Gestion des publications (CRUD Livres)

- Liste des livres avec statut (Brouillon, En attente de validation, Publié, Refusé, Archivé)
- Création d'un nouveau livre (formulaire multi-étapes) :

ÉTAPE 1 — Informations de base :

- Titre, sous-titre
- Description (éditeur WYSIWYG avec compteur de mots)
- Résumé court (max 500 caractères, pour SEO)
- Langue
- Catégorie(s) principale(s)
- Tags / Mots-clés (max 10)
- ISBN (optionnel, si déjà attribué)

ÉTAPE 2 — Fichiers :

- Upload couverture (guidelines : 1563x2500px, JPG/PNG, max 5Mo)
- Prévisualisation en temps réel
- Validation automatique des dimensions
- Upload ePub (max 50Mo)
- Validation du format (epubcheck)
- Extraction métadonnées (titre, auteur, langue depuis le ePub)
- Extraction nombre de pages
- Upload extrait gratuit (premiers chapitres, ePub séparé, max 5Mo)

ÉTAPE 3 — Prix et disponibilité :

- Prix de vente (en FCFA)
- Calcul automatique de la rémunération : “Vous recevrez 70% soit X FCFA par vente”
- Date de publication souhaitée
- Territoires de vente (par défaut : tous les pays éligibles)

ÉTAPE 4 — Récapitulatif et soumission :

- Prévisualisation complète de la fiche produit
- Checkbox : “Je certifie détenir les droits d’auteur”
- Bouton “Soumettre pour validation”
- Édition d’un livre publié (mise à jour de métadonnées, pas de fichier)
- Duplication d’un livre (pour créer une nouvelle édition)
- Archivage / Désarchivage
- Suppression (uniquement si 0 vente)

5.1.4 Statistiques détaillées par livre

Pour chaque livre, page dédiée avec :

- Vue d'ensemble :
  - Ventes totales et du mois en cours
  - Revenus générés
  - Nombre de pages lues (si lecteur intégré utilisé)
  - Taux de conversion (vues fiche → achats)
- Graphiques temporels :
  - Ventes par jour (30 derniers jours)
  - Ventes par mois (12 derniers mois)
  - Comparaison avec période précédente
- Tableau des ventes détaillées :
  - Date, client (anonymisé), pays, prix, commission, net
  - Export CSV/Excel
- Répartition géographique :
  - Ventes par pays (tableau + carte interactive)
  - Top 5 des pays
- Avis et notes :
  - Liste des avis avec réponse possible
  - Répartition des notes (1-5 étoiles)
  - Moyenne et évolution

5.1.5 Paiements et revenus

- Tableau des paiements reçus :
  - Date, période couverte, montant brut, commission (30%), net
  - Statut : Payé, En attente, Annulé
  - Historique complet (toutes les transactions)
  - Export pour comptabilité
- Informations de paiement :
  - Compte Mobile Money préféré
  - Historique des paiements

5.1.6 Communauté et ressources

- Centre d'aide : guides, tutoriels vidéo, FAQ
- Blog auteur : conseils d'écriture, marketing, promotion
- Forum / Communauté (optionnel Phase 2+)
- Support : formulaire de contact, chat (optionnel)

5.2 WORKFLOW DE PUBLICATION

5.2.1 Processus de validation

AUTEUR → Soumet livre → STATUT : “En attente de validation”→File d’attente admin→ADMIN examine (délai max 48h)→APPROUVÉ REFUSÉ DEMANDE MODIFS

STATUT : “Publié” STATUT : “Refusé” STATUT : “À modifier” Email auteur Email avec Email avec

- notification motifs de refus liste des corrections
- mise en ligne + possibilité + resoumission de révision

5.2.2 Critères de validation

- Qualité du fichier ePub (pas de corruption, format valide)
- Qualité de la couverture (dimensions, résolution, pas de contenu choquant)
- Contenu : pas de plagiat, pas de contenu illégal, pas de haine/ discrimination
- Métadonnées complètes et exactes
- Respect des droits d’auteur (l’auteur certifie être le détenteur des droits)
- Conformité aux conditions d’utilisation

5.2.3 Outils de validation automatique (pré-contrôle)

- Vérification ePub (epubcheck)
- Vérification dimensions couverture
- Vérification plagiat (outil tiers optionnel : Copyscape, Turnitin)
- Vérification contenu sensible (API de modération)
- Score de qualité avant soumission

5.3 TABLEAU DE BORD AUTEUR (ANALYTICS)

5.3.1 Vue d’ensemble (Dashboard principal)

Inspiré du dashboard Kobo Writing Life qui affiche une estimation des ventes en temps réel et des rapports mensuels détaillés.

Chiffres clés (cartes) :

- Ventes aujourd’hui
- Ventes ce mois
- Revenus ce mois (net après 30%)

- Livres en vente
- Nouveaux lecteurs ce mois
- Note moyenne globale
- Graphique principal : Ventes sur les 30 derniers jours (courbe)
- Graphique secondaire : Répartition des revenus par livre
- Tableau : Top 5 des livres les plus vendus ce mois
- Notifications : alertes récentes (ventes, avis, validations)

5.3.2 Rapports détaillés

Inspiré des rapports mensuels de Kobo avec export Excel et des analytics de Gumroad avec tracking par source et localisation.

Rapport de ventes :

- Période personnalisable (jour, semaine, mois, année, personnalisé)
- Ventes par livre (tableau détaillé)
- Ventes par pays (tableau + carte)
- Ventes par source (direct, réseaux sociaux, email, organique)
- Taux de conversion par source
- Export CSV / Excel / PDF

Rapport de revenus :

- Période, montant brut, commission plateforme (30%), montant net
- Répartition par livre
- Cumul annuel
- Projection (tendance)

Rapport de lecture (si lecteur intégré utilisé) :

- Pages lues par livre
- Temps de lecture moyen
- Taux d'achèvement
- Points d'abandon (chapitres où les lecteurs s'arrêtent)

5.3.3 Notifications en temps réel

WebSocket pour notifications instantanées :

- "Nouvelle vente ! [Titre du livre] — X FCFA"
- "Nouvel avis sur [Titre] — 4 étoiles"
- "Votre livre [Titre] a été approuvé et publié"
- "Votre paiement de X FCFA a été effectué"
- Historique des notifications (30 derniers jours)

- Paramètres : activer/désactiver par type, email vs push

5.4 SYSTÈME DE REVENUS ET PAIEMENTS

5.4.1 Calcul des revenus

Pour chaque vente :

- Prix de vente : P
- Commission plateforme : P × 30%
- Revenus auteur : P × 70%
- Arrondi à l'unité inférieure en FCFA

Exemple :

- Prix de vente : 2 000 FCFA
- Commission plateforme : 600 FCFA
- Revenus auteur : 1 400 FCFA

5.4.2 Paiement automatique mensuel

- Période de référence : du 1er au dernier jour du mois
- Calcul des revenus : le 1er du mois suivant
- Paiement : entre le 1er et le 5 du mois suivant
- Seuil minimum de paiement : 10 000 FCFA
  - Si seuil non atteint : report au mois suivant (cumul)
- Mode de paiement : Mobile Money (même numéro que l'inscription)
- Notification : email de confirmation avec détail du paiement
- Historique : tableau de tous les paiements reçus

5.4.3 Gestion des remboursements

- Politique : remboursement possible dans les 14 jours si e-book non téléchargé
- Impact sur revenus :
  - Si remboursement dans le mois de vente : déduction du montant net
  - Si remboursement après paiement : déduction du prochain paiement
- Journal des remboursements visible par l'auteur

5.4.4 Fiscalité

- L'auteur est responsable de ses obligations fiscales
- La plateforme fournit un récapitulatif annuel des revenus

- Pas de retenue à la source (sauf obligation légale future)

5.5 VALIDATION ÉDITORIALE (BACK-OFFICE ADMIN)

5.5.1 File de validation

Liste des livres en attente avec :

- Date de soumission, auteur, titre, catégorie
- Temps restant avant échéance (48h max)
- Alertes visuelles si > 24h, > 36h, > 48h
- Filtres : par date, par catégorie, par auteur
- Tri : plus ancien en premier

5.5.2 Interface de validation

Vue côte à côte :

- Gauche : prévisualisation de la fiche produit
- Droite : contrôles de validation

Contrôles :

- Vérification ePub (téléchargement + epubcheck)
- Vérification couverture (affichage + dimensions)
- Vérification métadonnées (complétude)
- Vérification contenu (extrait + scan anti-plagiat)

Actions :

- “Approuver et publier” → publication immédiate
- “Refuser” → formulaire avec motifs prédéfinis + commentaire libre
- “Demander des modifications” → liste de corrections à apporter
- “Reporter” → report avec raison

5.5.3 Motifs de refus prédéfinis

- Fichier ePub corrompu ou invalide
- Couverture de mauvaise qualité ou non conforme
- Contenu plagiaire ou non original
- Contenu illégal, haineux, discriminatoire, pornographique
- Métadonnées incomplètes ou incorrectes
- Violation des droits d’auteur
- Non-respect des conditions d’utilisation

5.5.4 Journal de validation

- Historique complet des validations/ refus
- Statistiques : temps moyen de validation, taux d'approbation
- Performance par modérateur

6. SPÉCIFICATIONS TECHNIQUES DÉTAILLÉES

6.1 MODÈLE DE DONNÉES COMPLET (PHASE 1 + PHASE 2)

6.1.1 Table USERS

CREATE TABLE users (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), email VARCHAR(255) NOT NULL UNIQUE, password_hash VARCHAR(255) NOT NULL, first_name VARCHAR(100) NOT NULL, last_name VARCHAR(100) NOT NULL, phone VARCHAR(20),

avatar_url VARCHAR(500), role VARCHAR(20) NOT NULL DEFAULT 'customer' CHECK (role IN ('customer', 'author', 'admin', 'super_admin')), is_active BOOLEAN DEFAULT TRUE, email_verified BOOLEAN DEFAULT FALSE, phone_verified BOOLEAN DEFAULT FALSE, kyc_status VARCHAR(20) DEFAULT 'not_required' CHECK (kyc_status IN ('not_required', 'pending', 'approved', 'rejected')), kyc_document_url VARCHAR(500), preferred_payment_phone VARCHAR(20), cre-ated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), deleted_at TIMESTAMP WITH TIME ZONE – Soft delete);

CREATE INDEX idx_users_email ON users(email); CREATE INDEX idx_users_role ON users(role); CREATE INDEX idx_users_created_at ON users(created_at);

6.1.2 Table AUTHORS (profil auteur, Phase 2)

CREATE TABLE authors (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL UNIQUE REFERENCES users(id) ON DELETE CASCADE, pen_name VARCHAR(200),

bio TEXT, photo_url VARCHAR(500), website VARCHAR(500), social_links JSONB DEFAULT '{}',

is_verified BOOLEAN DEFAULT FALSE, total_books INTEGER DEFAULT 0, total_sales INTEGER DEFAULT 0, total_revenue DECIMAL(12,2) DEFAULT 0, created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW());

6.1.3 Table BOOKS

CREATE TABLE books (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), title VARCHAR(500) NOT NULL, slug VARCHAR(600) NOT NULL UNIQUE, subtitle VARCHAR(500),

description TEXT, short_description VARCHAR(500), price DECIMAL(10,2) NOT NULL CHECK (price > 0), currency VARCHAR(3) DEFAULT 'XOF', isbn VARCHAR(13), language VARCHAR(10) DEFAULT 'fr', page_count INTEGER, cover_url VARCHAR(500), file_url VARCHAR(500) NOT NULL, file_size INTEGER, – en octets excerpt_url VARCHAR(500), excerpt_size INTEGER, status VARCHAR(20) NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'pending', 'published', 'rejected', 'archived')),

author_id UUID REFERENCES authors(id) ON DELETE SET NULL, cat-egory_id UUID REFERENCES categories(id) ON DELETE SET NULL, is_featured BOOLEAN DEFAULT FALSE, published_at TIMESTAMP WITH TIME ZONE, submitted_at TIMESTAMP WITH TIME ZONE, – Date de soumission pour validation validated_at TIMESTAMP WITH TIME ZONE, – Date de validation validated_by UUID REFERENCES users(id) ON DELETE SET NULL, rejection_reason TEXT, sales_count INTEGER DEFAULT 0, rating_avg DECIMAL(2,1) DEFAULT 0 CHECK (rating_avg >= 0 AND rating_avg <= 5), rating_count INTEGER DEFAULT 0, view_count INTEGER DEFAULT 0, download_count INTEGER DEFAULT 0, created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), created_by UUID REFERENCES users(id) ON DELETE SET NULL);

CREATE INDEX idx_books_slug ON books(slug);CREATE INDEX idx_books_status ON books(status); CREATE INDEX idx_books_author ON books(author_id); CREATE INDEX idx_books_category ON books(category_id); CREATE INDEX idx_books_published ON books(published_at); CREATE INDEX idx_books_featured ON books(is_featured) WHERE is_featured = TRUE; CREATE INDEX idx_books_price ON books(price);

– Index pour recherche full-text CREATE INDEX idx_books_search ON books USING GIN (to_tsvector('french', coalesce(title,'') ||' ' || coalesce(description,' ')));

6.1.4 Table CATEGORIES

CREATE TABLE categories (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), name VAR-CHAR(200) NOT NULL, slug VARCHAR(250) NOT NULL UNIQUE, description TEXT, parent_id UUID REFERENCES categories(id) ON DELETE SET NULL, image_url VARCHAR(500), sort_order INTEGER DEFAULT 0, is_active BOOLEAN DEFAULT TRUE, book_count INTEGER DEFAULT 0, created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW());

CREATE INDEX idx_categories_slug ON categories(slug); CREATE INDEX idx_categories_parent ON categories(parent_id);

6.1.5 Table BOOK_CATEGORIES (association N-N)

CREATE TABLE book_categories (book_id UUID REFERENCES books(id) ON DELETE CAS-CADE, category_id UUID REFERENCES categories(id) ON DELETE CASCADE, is_primary BOOLEAN DEFAULT FALSE, PRIMARY KEY (book_id, category_id));

6.1.6 Table ORDERS

CREATE TABLE orders (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, status VARCHAR(20) NOT NULL

DEFAULT 'pending' CHECK (status IN ('pending', 'paid', 'processing', 'completed', 'cancelled', 're-funded')), total_amount DECIMAL(10,2) NOT NULL CHECK (total_amount >= 0), currency VAR-CHAR(3) DEFAULT 'XOF', coupon_id UUID REFERENCES coupons(id) ON DELETE SET NULL,

discount_amount DECIMAL(10,2) DEFAULT 0, payment_id UUID REFERENCES payments(id) ON DELETE SET NULL, customer_email VARCHAR(255) NOT NULL, customer_phone VARCHAR(20),

customer_name VARCHAR(200), billing_address JSONB, ip_address INET, user_agent TEXT, cre-ated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), completed_at TIMESTAMP WITH TIME ZONE);

CREATE INDEX idx_orders_user ON orders(user_id); CREATE INDEX idx_orders_status ON orders(status); CREATE INDEX idx_orders_created_at ON orders(created_at); CREATE INDEX idx_orders_payment ON orders(payment_id);

6.1.7 Table ORDER_ITEMS

CREATE TABLE order_items (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE, book_id UUID NOT NULL REFERENCES books(id) ON DELETE CASCADE, quantity INTEGER NOT NULL DEFAULT 1

CHECK (quantity > 0), unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0), to-tal_price DECIMAL(10,2) NOT NULL CHECK (total_price >= 0), author_royalty DECIMAL(10,2) NOT NULL, – 70% du prix platform_fee DECIMAL(10,2) NOT NULL, – 30% du prix created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW());

CREATE INDEX idx_order_items_order ON order_items(order_id); CREATE INDEX idx_order_items_book ON order_items(book_id);

6.1.8 Table PAYMENTS

CREATE TABLE payments (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE, provider VARCHAR(50)

NOT NULL CHECK (provider IN ('orange_money', 'mtn_money', 'wave', 'moov_money', 'stripe')), provider_ref VARCHAR(255), – Référence externe de l'opérateur amount DECIMAL(10,2) NOT NULL CHECK (amount >= 0), currency VARCHAR(3) DEFAULT 'XOF', status VARCHAR(20) NOT NULL

DEFAULT 'pending' CHECK (status IN ('pending', 'processing', 'completed', 'failed', 'refunded')), phone_number VARCHAR(20) NOT NULL, provider_response JSONB, – Réponse brute de l'API opérateur webhook_data JSONB, – Données du webhook paid_at TIMESTAMP WITH TIME ZONE, refunded_at TIMESTAMP WITH TIME ZONE, retry_count INTEGER DEFAULT 0, created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW());

CREATE INDEX idx_payments_order ON payments(order_id); CREATE INDEX idx_payments_provider ON payments(provider); CREATE INDEX idx_payments_status ON payments(status); CREATE INDEX idx_payments_created_at ON payments(created_at);

6.1.9 Table COUPONS

CREATE TABLE coupons (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), code VAR-CHAR(50) NOT NULL UNIQUE, type VARCHAR(20) NOT NULL CHECK (type IN ('percentage', 'fixed')), value DECIMAL(10,2) NOT NULL CHECK (value >= 0), min_amount DECIMAL(10,2) DE-FAULT 0, max_discount DECIMAL(10,2), – Limite pour les pourcentages max_uses INTEGER, – NULL = illimité used_count INTEGER DEFAULT 0, valid_from TIMESTAMP WITH TIME ZONE NOT NULL, valid_until TIMESTAMP WITH TIME ZONE NOT NULL, is_active BOOLEAN DEFAULT

TRUE, created_by UUID REFERENCES users(id) ON DELETE SET NULL, created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW());

CREATE INDEX idx_coupons_code ON coupons(code); CREATE INDEX idx_coupons_validity ON coupons(valid_from, valid_until);

6.1.10 Table REVIEWS

CREATE TABLE reviews (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), book_id UUID NOT NULL REFERENCES books(id) ON DELETE CASCADE, user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, rating INTEGER NOT NULL CHECK (rating >= 1 AND rating <= 5), comment TEXT, is_approved BOOLEAN DEFAULT FALSE, is_featured BOOLEAN DEFAULT FALSE, helpful_count INTEGER DEFAULT 0, created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), updated_at TIMESTAMP WITH TIME ZONE DEFAULT

NOW(), UNIQUE(book_id, user_id) – Un avis par utilisateur par livre);

CREATE INDEX idx_reviews_book ON reviews(book_id); CREATE INDEX idx_reviews_user ON reviews(user_id); CREATE INDEX idx_reviews_approved ON reviews(is_approved); CREATE INDEX idx_reviews_rating ON reviews(rating);

6.1.11 Table LIBRARY (achats client)

CREATE TABLE library (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, book_id UUID NOT NULL REFER-ENCES books(id) ON DELETE CASCADE, order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE, download_count INTEGER DEFAULT 0 CHECK (download_count <= 5), max_downloads INTEGER DEFAULT 5, last_download TIMESTAMP WITH TIME ZONE, created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), UNIQUE(user_id, book_id));

CREATE INDEX idx_library_user ON library(user_id); CREATE INDEX idx_library_book ON li-brary(book_id);

6.1.12 Table READING_PROGRESS

CREATE TABLE reading_progress (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, book_id UUID NOT NULL REFERENCES books(id) ON DELETE CASCADE, cfi VARCHAR(500) NOT NULL, – Canon-

ical Fragment Identifier progress_percent DECIMAL(5,2) DEFAULT 0 CHECK (progress_percent >= 0 AND progress_percent <= 100), last_read TIMESTAMP WITH TIME ZONE DEFAULT NOW(), created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), updated_at TIMESTAMP WITH

TIME ZONE DEFAULT NOW(), UNIQUE(user_id, book_id));

CREATE INDEX idx_reading_progress_user ON reading_progress(user_id);

6.1.13 Table BOOKMARKS

CREATE TABLE bookmarks (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID

NOT NULL REFERENCES users(id) ON DELETE CASCADE, book_id UUID NOT NULL REFERENCES books(id) ON DELETE CASCADE, cfi VARCHAR(500) NOT NULL, note TEXT, color VARCHAR(20) DEFAULT 'yellow', created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW());

CREATE INDEX idx_bookmarks_user_book ON bookmarks(user_id, book_id);

6.1.14 Table AUTHOR_PAYMENTS (Phase 2)

CREATE TABLE author_payments (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), author_id UUID NOT NULL REFERENCES authors(id) ON DELETE CASCADE, period_start DATE NOT NULL, period_end DATE NOT NULL, total_sales INTEGER NOT NULL DEFAULT 0, gross_revenue DECIMAL(12,2) NOT NULL DEFAULT 0, platform_fee DECIMAL(12,2) NOT NULL DEFAULT 0, net_revenue DECIMAL(12,2) NOT NULL DEFAULT 0, status VARCHAR(20)

NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'processing', 'paid', 'failed')), payment_method VARCHAR(50) DEFAULT 'mobile_money', payment_phone VARCHAR(20), transaction_ref VARCHAR(255), paid_at TIMESTAMP WITH TIME ZONE, created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(), updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW());

CREATE INDEX idx_author_payments_author ON author_payments(author_id); CREATE INDEX idx_author_payments_period ON author_payments(period_start, period_end); CREATE INDEX idx_author_payments_status ON author_payments(status);

6.1.15 Table AUDIT_LOGS

CREATE TABLE audit_logs (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID REFERENCES users(id) ON DELETE SET NULL, action VARCHAR(100) NOT NULL, – e.g.,

'book.created', 'order.completed' entity_type VARCHAR(50) NOT NULL, – 'book', 'order', 'user', etc. entity_id UUID, old_values JSONB, new_values JSONB, ip_address INET, user_agent TEXT, cre-ated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW());

CREATE INDEX idx_audit_logs_user ON audit_logs(user_id); CREATE INDEX idx_audit_logs_action ON audit_logs(action); CREATE INDEX idx_audit_logs_entity ON audit_logs(entity_type, entity_id); CREATE INDEX idx_audit_logs_created ON audit_logs(created_at);

6.2 API REST — ENDPOINTS DÉTAILLÉS

6.2.1 Authentification

POST /api/v1/auth/register Body: { email, password, first_name, last_name, phone } Response: { user, access_token, refresh_token }

POST /api/v1/auth/login Body: { email, password } Response: { user, access_token, refresh_token } POST

/api/v1/auth/refresh Body: { refresh_token } Response: { access_token, refresh_token }

POST /api/v1/auth/logout Headers: Authorization: Bearer {access_token} Response: { success: true }

POST /api/v1/auth/forgot-password Body: { email } Response: { message: "Email envoyé" }

POST /api/v1/auth/reset-password Body: { token, new_password } Response: { success: true } POST /api/v1/auth/verify-email Body: { token } Response: { success: true }

GET /api/v1/auth/me Headers: Authorization: Bearer {access_token} Response: { user }

PUT /api/v1/auth/me Headers: Authorization: Bearer {access_token} Body: { first_name, last_name, phone, avatar_url } Response: { user }

6.2.2 Livres (Catalogue Public)

GET /api/v1/books Query: { page, limit, category, min_price, max_price, rating, sort_by, sort_order, search, language } Response: { items: [Book], total, page, limit, total_pages }

GET /api/v1/books/{slug} Response: { Book (détail complet avec auteur, catégories, avis) } GET /api/v1/books/{slug}/related Query: { limit } Response: { items: [Book] }

GET /api/v1/books/search Query: { q, page, limit, filters } Response: { items: [Book], total, facets } – Recherche full-text PostgreSQL avec tsvector

GET /api/v1/categories Response: { items: [Category] } – Arborescence complète GET /api/v1/categories/{slug} Response: { Category avec livres associés }

GET /api/v1/categories/{slug}/books Query: { page, limit, sort_by } Response: { items: [Book], total }

6.2.3 Avis

GET /api/v1/books/{slug}/reviews Query: { page, limit, sort_by } Response: { items: [Review], total, avg_rating }

POST /api/v1/books/{slug}/reviews Headers: Authorization: Bearer {access_token} Body: { rating, comment } Response: { review } – Un avis par utilisateur par livre

PUT /api/v1/reviews/{id} Headers: Authorization: Bearer {access_token} Body: { rating, comment } Response: { review } – Seulement l'auteur de l'avis

DELETE /api/v1/reviews/{id} Headers: Authorization: Bearer {access_token} Response: { success: true } – Auteur de l'avis ou admin

6.2.4 Panier

GET /api/v1/cart Headers: Authorization: Bearer {access_token} (optionnel) Response: { items: [CartItem], total, item_count } – Si non authentifié, lit depuis localStorage (cookie session)

POST /api/v1/cart/items Headers: Authorization: Bearer {access_token} (optionnel) Body: { book_id, quantity } Response: { cart }

PUT /api/v1/cart/items/{id} Headers: Authorization: Bearer {access_token} Body: { quantity } Response: { cart }

DELETE /api/v1/cart/items/{id} Headers: Authorization: Bearer {access_token} Response: { cart }

POST /api/v1/cart/apply-coupon Headers: Authorization: Bearer {access_token} Body: { code } Response: { cart, discount }

DELETE /api/v1/cart/clear Headers: Authorization: Bearer {access_token} Response: { cart }

6.2.5 Commandes

POST /api/v1/orders Headers: Authorization: Bearer {access_token} Body: { cart_id, customer_email, customer_phone, customer_name, billing_address, coupon_code } Response: { order, payment_url } – Crée la commande et initie le paiement

GET /api/v1/orders Headers: Authorization: Bearer {access_token} Query: { page, limit, status } Response: { items: [Order], total }

GET /api/v1/orders/{id} Headers: Authorization: Bearer {access_token} Response: { Order (détail complet) }

POST /api/v1/orders/{id}/cancel Headers: Authorization: Bearer {access_token} Response: { order } – Annulation possible si statut = pending

GET /api/v1/orders/{id}/invoice Headers: Authorization: Bearer {access_token} Response: PDF (fac-ture)

6.2.6 Paiements

POST /api/v1/payments/initiate Headers: Authorization: Bearer {access_token} Body: { order_id, provider, phone_number } Response: { payment, instructions } – Initie le paiement Mobile Money

POST /api/v1/payments/webhook/{provider} Headers: X-Signature (HMAC) Body: { provider_ref, status, metadata } Response: { success: true } – Webhook appelé par l'opérateur Mobile Money

GET /api/v1/payments/{id}/status Headers: Authorization: Bearer {access_token} Response: { pay-ment }

POST /api/v1/payments/{id}/retry Headers: Authorization: Bearer {access_token} Response: { pay-ment } – Retry si échec précédent

6.2.7 Bibliothèque (Library)

GET /api/v1/library Headers: Authorization: Bearer {access_token} Query: { page, limit, sort_by } Response: { items: [LibraryItem], total } – Liste des e-books achetés

GET /api/v1/library/{book_id}/download Headers: Authorization: Bearer {access_token} Response: { download_url, expires_at, downloads_remaining } – URL signée temporaire (15 min), max 5 télécharge-ments

GET /api/v1/library/{book_id}/read Headers: Authorization: Bearer {access_token} Response: { read_url, book_metadata } – URL pour le lecteur intégré

6.2.8 Progression de lecture

GET /api/v1/library/{book_id}/progress Headers: Authorization: Bearer {access_token} Response: { progress }

PUT /api/v1/library/{book_id}/progress Headers: Authorization: Bearer {access_token} Body: { cfi, progress_percent } Response: { progress }

6.2.8 Marque-pages

GET /api/v1/library/{book_id}/bookmarks Headers: Authorization: Bearer {access_token} Response:

{ items: [Bookmark] }

POST /api/v1/library/{book_id}/bookmarks Headers: Authorization: Bearer {access_token} Body: { cfi, note, color } Response: { bookmark }

DELETE /api/v1/library/bookmarks/{id} Headers: Authorization: Bearer {access_token} Response: { success: true }

6.2.9 Admin — Gestion du catalogue

GET /api/v1/admin/books Headers: Authorization: Bearer {access_token} (role: admin) Query: { page, limit, status, search } Response: { items: [Book], total }

POST /api/v1/admin/books Headers: Authorization: Bearer {access_token} Body: { title, subtitle, description, price, category_id, author_id, isbn, language, page_count, cover, file, excerpt } Response:

{ book } – Upload multipart/form-data

PUT /api/v1/admin/books/{id} Headers: Authorization: Bearer {access_token} Body: { …champs modifiables… } Response: { book }

DELETE /api/v1/admin/books/{id} Headers: Authorization: Bearer {access_token} Response: { suc-cess: true } – Soft delete

6.2.10 Admin — Gestion des commandes

GET /api/v1/admin/orders Headers: Authorization: Bearer {access_token} Query: { page, limit, status, date_from, date_to, min_amount } Response: { items: [Order], total, stats }

GET /api/v1/admin/orders/{id} Headers: Authorization: Bearer {access_token} Response: { Order (détail complet avec historique) }

PUT /api/v1/admin/orders/{id}/status Headers: Authorization: Bearer {access_token} Body: { status, reason } Response: { order }

POST /api/v1/admin/orders/{id}/refund Headers: Authorization: Bearer {access_token} Body: { amount, reason } – amount optionnel pour remboursement partiel Response: { order, refund }

6.2.11 Admin — Rapports

GET /api/v1/admin/reports/sales Headers: Authorization: Bearer {access_token} Query: { period, group_by, format } Response: { data } ou fichier (CSV/PDF)

GET /api/v1/admin/reports/revenue Headers: Authorization: Bearer {access_token} Query: { period, format } Response: { data } ou fichier

GET /api/v1/admin/reports/customers Headers: Authorization: Bearer {access_token} Query: { pe-riod, format } Response: { data } ou fichier

GET /api/v1/admin/dashboard/stats Headers: Authorization: Bearer {access_token} Response: { kpi_cards,

charts } – Données agrégées pour le dashboard admin

6.2.12 Auteur — Gestion des publications (Phase 2)

GET /api/v1/author/books Headers: Authorization: Bearer {access_token} (role: author) Query: { page, limit, status } Response: { items: [Book], total }

POST /api/v1/author/books Headers: Authorization: Bearer {access_token} Body: { title, subtitle, description, price, category_id, tags, isbn, language, cover, file, excerpt } Response: { book } – Création en statut “draft”

PUT /api/v1/author/books/{id} Headers: Authorization: Bearer {access_token} Body: { …champs modifiables… } Response: { book } – Uniquement si auteur = propriétaire

POST /api/v1/author/books/{id}/submit Headers: Authorization: Bearer {access_token} Response: { book } – Soumet pour validation (statut -> pending)

DELETE /api/v1/author/books/{id} Headers: Authorization: Bearer {access_token} Response: { success: true } – Uniquement si 0 vente

6.2.13 Auteur — Statistiques (Phase 2)

GET /api/v1/author/stats Headers: Authorization: Bearer {access_token} Query: { period } Response: { total_sales, total_revenue, books_count, avg_rating, charts }

GET /api/v1/author/books/{id}/stats Headers: Authorization: Bearer {access_token} Query: { period } Response: { sales, revenue, views, downloads, conversion_rate, geographic_data }

GET /api/v1/author/payments Headers: Authorization: Bearer {access_token} Query: { page, limit, status } Response: { items: [AuthorPayment], total }

GET /api/v1/author/payments/{id} Headers: Authorization: Bearer {access_token} Response: { AuthorPayment (détail avec transactions) }

6.2.14 Admin — Validation éditoriale (Phase 2)

GET /api/v1/admin/validation/queue Headers: Authorization: Bearer {access_token} (role: admin/ moderator) Query: { page, limit, status, overdue } Response: { items: [Book], total, overdue_count }

GET /api/v1/admin/validation/books/{id} Headers: Authorization: Bearer {access_token} Response: { Book (détail complet pour validation) }

POST /api/v1/admin/validation/books/{id}/approve Headers: Authorization: Bearer {access_token} Body: { notes } Response: { book } – Publie le livre

POST /api/v1/admin/validation/books/{id}/reject Headers: Authorization: Bearer {access_token} Body: { reason, rejection_details } Response: { book }

POST / api/ v1/ admin/ validation/ books/{id}/ request-changes Headers: Authorization: Bearer {access_token} Body: { changes_list, notes } Response: { book }

GET / api/ v1/ admin/ validation/ stats Headers: Authorization: Bearer {access_token} Response: { avg_validation_time, approval_rate, pending_count, overdue_count }

6.3 INTÉGRATIONS TIERCES

6.3.1 Mobile Money APIs

- Orange Money API (Orange Developer Portal)
  - Endpoint : https://api.orange.com/orange-money/dev/v1
  - Authentification : OAuth 2.0 client credentials
  - Flow : Payment Request → Customer Validation → Webhook Confirmation
  - Documentation : developer.orange.com
- MTN Mobile Money API (MTN MoMo)
  - Endpoint : https://sandbox.momodeveloper.mtn.com (dev)
  - Authentification : API Key + Subscription Key
  - Flow : RequestToPay → Customer PIN → Webhook
  - Documentation : momodeveloper.mtn.com
- Wave API
  - Endpoint : https://api.wave.com/v1
  - Authentification : API Key
  - Flow : Checkout URL → Customer Payment → Webhook
  - Documentation : developer.wave.com
- Moov Money API
  - Endpoint : selon pays (Côte d'Ivoire, Togo)
  - Authentification : API Key
  - Flow : similaire à Orange Money

6.3.2 Stockage de fichiers (S3-compatible)

- Cloudflare R2 (recommandé pour l'Afrique)
  - Pas de frais de sortie (egress)
  - Compatible API S3
  - CDN intégré
  - Configuration : R2_ACCESS_KEY_ID, R2_SECRET_ACCESS_KEY, R2_ENDPOINT_URL, R2_BUCKET_NAME
- Alternative : AWS S3 + CloudFront
  - Plus mature mais frais de sortie élevés
  - CloudFront Edge Locations en Afrique (Johannesburg, Nairobi)

6.3.3 Email (Transactionnels)

- Resend (recommandé)
  - Domaine personnalisé obligatoire
  - Templates HTML avec variables
  - Webhooks pour tracking (ouverture, clic, bounce)

- Tarif : 3 000 emails/mois gratuits, puis $0.0001/email
- Alternative : Brevo (ex-Sendinblue)
  - 300 emails/jour gratuits
  - Templates drag & drop
  - API REST simple

6.3.4 Monitoring et Observabilité

- Sentry (erreurs)
  - Intégration FastAPI via sentry-sdk
  - Tracking des erreurs 500, timeouts, exceptions
  - Alertes Slack/Email
- Grafana + Prometheus (métriques)
  - Métriques custom : ventes, paiements, inscriptions
  - Dashboards : performance API, business metrics
  - Alertes sur seuils (ex: taux d'erreur > 1%)

6.3.5 CI/CD

- GitHub Actions
  - Workflow : test → build → deploy
  - Tests : pytest (backend), jest (frontend)
  - Linting : ruff, black (Python), ESLint (TypeScript)
  - Déploiement : Railway/Render (staging), AWS ECS (prod)

6.4 PERFORMANCE & SCALABILITÉ

6.4.1 Cache

- Redis pour :
  - Sessions utilisateur (TTL : 7 jours)
  - Cache des pages (Next.js ISR, revalidation 60s)
  - Cache des requêtes API (TTL : 5 min pour catalogue, 1h pour catégories)
  - Rate limiting (sliding window)
  - File d'attente Celery
- Stratégie de cache :
  - Cache-First pour catalogue public
  - Stale-While-Revalidate pour données fréquemment consultées
  - Invalidation explicite sur mutation (CRUD admin)

6.4.2 Base de données

- Connection pooling : PgBouncer (max 100 connexions)
- Requêtes optimisées :
  - SELECT avec LIMIT/OFFSET (pas de OFFSET > 10 000)

- Curseurs pour pagination infinie (cursor-based)
- Index couvrants pour requêtes fréquentes • Partitionnement (Phase 2) :
- orders : par mois (range partitioning)
- payments : par mois
- audit_logs : par mois

6.4.3 CDN et assets

- Cloudflare CDN
  - Cache des images (couvertures) : 1 an
  - Cache des fichiers statiques (JS, CSS) : 1 an avec hash
  - Cache API (GET public) : 5 min
  - Purge automatique sur déploiement

6.4.4 Upload de fichiers

- Chunked upload pour fichiers > 10 Mo
- Validation côté client (type, taille) avant upload
- Compres-sion images (Sharp) : WebP, max 1200px de large
- Scan antivirus (ClamAV) pour les ePub

6.4.5 Scalabilité horizontale

- Backend : stateless (pas de session en mémoire)
- File d'attente : Celery workers horizontaux
- Base de données : read replicas (Phase 2)
- Stockage : S3/R2 (infiniment scalable)

7. EXIGENCES NON-FONCTIONNELLES

7.1 Disponibilité

- SLA : 99.9% (max 43 min d'indisponibilité/mois)
- Maintenance planifiée : fenêtre 02:00-04:00 UTC, notification 48h avant
- Backup automatique : quotidien (DB), continu (fichiers S3 versioning)
- Plan de reprise d'activité (PRA) : RTO < 4h, RPO < 1h

7.2 Performance

- Temps de chargement page : < 2s (Lighthouse Performance > 90)
- Temps de réponse API : < 200ms (p95), < 500ms (p99)
- Taux d'erreur : < 0.1%

- Concurrency : 10 000 utilisateurs simultanés (Phase 1)

7.3 Sécurité

- OWASP Top 10 compliance
- Penetration testing annuel
- Audit de sécurité avant chaque mise en production majeure
- Chiffrement des données sensibles en base (PII)
- Rotation des secrets (API keys, JWT secrets) tous les 90 jours

7.4 Accessibilité

- WCAG 2.1 AA compliance
- Support lecteurs d'écran (ARIA labels)
- Contraste suffisant (ratio 4.5:1 minimum)
- Navigation au clavier complète

7.5 Internationalisation (i18n)

- Langue par défaut : Français
- Support multilingue : structure prête pour ajout de langues
- Devises : FCFA (XOF) par défaut, conversion automatique
- Fuseaux horaires : UTC stockage, affichage locale du client

7.6 Compatibilité navigateurs

- Chrome/Edge (2 dernières versions)
- Firefox (2 dernières versions)
- Safari (2 dernières versions)
- Mobile : iOS Safari, Chrome Android

8. LIVRABLES ET PLANNING

8.1 Phase 1 — MVP E-commerce (3-4 mois)

Semaine	Livrable	Responsable
S1-S2	Architecture, setup CI/CD, DB, auth	Backend
S2-S3	API catalogue (livres, catégories, recherche)	Backend
S3-S4	API panier, commandes, paiements	Backend
S4-S5	API bibliothèque, lecteur ePub	Backend
S5-S6	Frontend : page d'accueil, catalogue, fiche	Frontend
S6-S7	Frontend : panier, checkout, paiement	Frontend

S7-S8	Frontend : espace client, bibliothèque	Frontend
S8-S9	Frontend : lecteur ePub, pages statiques	Frontend
S9-S10	Back-office admin : dashboard, catalogue	Fullstack
S10-S11	Back-office admin : commandes, clients, avis	Fullstack
S11-S12	Back-office admin : rapports, paramètres	Fullstack
S12-S13	Tests E2E, optimisation, bug fixing	QA
S13-S14	Déploiement production, monitoring	DevOps
S14-S16	Beta fermée, retours utilisateurs, ajustements	Produit

8.2 Phase 2 — Auto-publication (4-5 mois après Phase 1)

Semaine	Livrable	Responsable
S1-S2	API auteur : inscription, profil, KYC	Backend
S2-S3	API auteur : CRUD livres, soumission	Backend
S3-S4	API auteur : statistiques, rapports	Backend
S4-S5	API admin : validation éditoriale	Backend
S5-S6	API : système de paiements auteurs	Backend
S6-S7	Frontend auteur : dashboard, publications	Frontend
S7-S8	Frontend auteur : statistiques, revenus	Frontend
S8-S9	Frontend admin : file de validation	Frontend
S9-S10	Frontend admin : gestion paiements auteurs	Frontend
S10-S12	Tests, optimisation, documentation	QA
S12-S14	Déploiement, onboarding premiers auteurs	Produit

8.3 Livrables documentaires

- Documentation API (OpenAPI/Swagger)
- Guide d'installation et déploiement (README, Docker)
- Guide d'utilisation admin
- Guide d'utilisation auteur
- Documentation technique (architecture, choix techniques)
- Tests automatisés (couverture > 80%)

9. GLOSSAIRE

- **API** : Application Programming Interface — interface de programmation
- **CDN** : Content Delivery Network — réseau de distribution de contenu
- **CRUD** : Create, Read, Update, Delete — opérations de base sur données
- **ePub** : Electronic Publication — format standard d'e-book ouvert
- **FCFA** : Franc CFA — devise commune de l'Afrique de l'Ouest et centrale
- **JWT** : JSON Web Token — token signé pour authentification stateless

- **KYC** : Know Your Customer — vérification d'identité
- **MVP** : Minimum Viable Product — produit minimum viable
- **ORM** : Object-Relational Mapping — mapping objet-relationnel
- **PWA** : Progressive Web App — application web progressive
- **RGPD** : Règlement Général sur la Protection des Données (UE)
- **SEO** : Search Engine Optimization — optimisation pour moteurs de recherche
- **SLA** : Service Level Agreement — contrat de niveau de service
- **SSR** : Server-Side Rendering — rendu côté serveur
- **URL signée** : URL avec token temporaire d'accès autorisé
- **WAF** : Web Application Firewall — pare-feu applicatif web
- **WebSocket** : Protocole de communication bidirectionnelle en temps réel
- **WYSIWYG** : What You See Is What You Get — éditeur visuel

ANNEXES

ANNEXE A : STACK TECHNIQUE DÉTAILLÉE

- Frontend :
  - Next.js 14 (App Router)
  - React 18 + TypeScript 5
  - Tailwind CSS 3.4
  - shadcn/ui (composants)
  - TanStack Query (React Query) 5
  - Zustand 4 (état global)
  - EPUB.js (lecteur ePub)
- React Hook Form + Zod (formulaires) Backend :
  - Python 3.11+
  - FastAPI 0.110+ (ou Django 5 + DRF)
  - SQLAlchemy 2.0 (ou Django ORM)
  - Alembic (migrations)
  - Pydantic 2 (validation)
  - Celery 5 + Redis 7
  - pytest (tests)
  - Ruff + Black (linting/formatting)
- Infrastructure :
  - Docker + Docker Compose
  - PostgreSQL 15
  - Redis 7
  - Cloudflare R2 (stockage)
  - Cloudflare CDN + WAF
  - Railway / Render / AWS ECS
  - GitHub Actions (CI/CD)
  - Sentry (monitoring erreurs)
  - Grafana + Prometheus (métriques)

ANNEXE B : MODÈLE DE DONNÉES VISUEL (DIAGRAMME ER)

[Voir section 6.1 pour le schéma complet avec CREATE TABLE] Relations principales :

- users (1) → (N) orders
- users (1) → (1) authors (si role = ‘author’)
- authors (1) → (N) books
- books (N) → (N) categories (via book_categories)

- books (1) → (N) reviews
- books (1) → (N) order_items
- orders (1) → (N) order_items
- orders (1) → (1) payments
- users (1) → (N) library
- users (1) → (N) reading_progress
- users (1) → (N) bookmarks
- authors (1) → (N) author_payments

ANNEXE C : FLUX DE PAIEMENT MOBILE MONEY (SÉQUENCE)

1. CLIENT : Sélectionne “Payer par Mobile Money” + saisit numéro
2. FRONTEND : POST /api/v1/payments/initiate { order_id, provider, phone }
3. BACKEND : Crée payment (status: pending) → appelle API opérateur
4. OPÉRATEUR : Envoie push/SMS au client pour validation
5. CLIENT : Valide le paiement sur son téléphone (PIN/confirmation)
6. OPÉRATEUR : Traite le paiement → envoie webhook à la plateforme
7. BACKEND : Reçoit webhook → met à jour payment (status: completed) → met à jour order (status: paid) → ajoute livres à la bibliothèque du client → envoie email de confirmation → déclenche tâches Celery (rapports, analytics)
8. CLIENT : Reçoit confirmation + accès à la bibliothèque

ANNEXE D : CALCUL DE RÉMUNÉRATION AUTEUR

Formule : Revenu net auteur = (Prix de vente × Quantité vendue) × 70% Commission plateforme = (Prix de vente × Quantité vendue) × 30%

Exemple pour un livre à 2 500 FCFA, 100 ventes : Revenu brut = 2 500 × 100 = 250 000 FCFA Commission plateforme = 250 000 × 30% = 75 000 FCFA Revenu net auteur = 250 000 × 70% = 175 000 FCFA

Paiement mensuel (si seuil atteint) :

- Période : 1er-31 du mois M
- Calcul : 1er du mois M+1
- Paiement : 1er-5 du mois M+1
- Seuil minimum : 10 000 FCFA

ANNEXE E : CHECKLIST DE SÉCURITÉ

HTTPS obligatoire (redirect HTTP → HTTPS) Headers de sécurité (HSTS, CSP, X-Frame-Options, X-Content-Type-Options) JWT sécurisés (access 15min, refresh 7j, HttpOnly, Secure, SameSite) Rate limiting (100 req/min IP, 1000 req/min auth) Validation des entrées (Pydantic, sanitize HTML) Protection XSS (échappement, CSP) Protection CSRF (tokens pour mutations) Protection injection SQL (ORM, requêtes paramétrées) Hashage mots de passe (Argon2id) Upload sécurisé (type MIME, taille, scan antivirus) URLs signées pour fichiers (expiration, limite) Journalisation sécurité (tentatives connexion, anomalies) RGPD compliance (consentement, droit à l’effacement, portabilité) Audit logs (toutes les actions administratives)

Rotation des secrets (90 jours) Backup chiffré (DB + fichiers)

FIN DU CAHIER DES CHARGES

Document rédigé le 03 Août 2026 Version 1.0

Auteur : AHOULOUMI Essowaba

DG De KEFALE SOFTWARE INNOVATION